

# Szoftvertervezési minták bemutatása

## Amőba (Gomoku) Java alkalmazás

Név: Marsi Gergő

Neptun kód: I9J2WL

---

## Bevezetés

Egy szoftver nem csupán attól tekinthető jónak, hogy működik, hanem attól is, hogy mennyire átlátható, könnyen módosítható és tesztelhető. A szoftverfejlesztés során ezért gyakran alkalmazunk olyan bevált megoldásokat, amelyek segítenek ezeknek a szempontoknak megfelelni. Ezeket a megoldásokat nevezzük szoftvertervezési mintáknak.

Az Amőba (Gomoku) Java projekt egy parancssoros játék, amelyben egy emberi játékos és egy számítógép játszik egymás ellen. A projekt fejlesztése során több olyan szerkezeti és architektúráis megoldás jelent meg, amelyek jól illeszkednek a tervezési minták szemléletéhez. Bár ezek egy része nem formális módon került bevezetésre, mégis jelentősen hozzájárulnak a kód tisztaságához és bővíthetőségéhez.

A dokumentum célja, hogy bemutassa a projektben felismerhető tervezési mintákat és alapelveket, konkrét példákon keresztül.

---

## 1. Model–View–Controller (MVC) architektúráis megközelítés

### Az MVC alapelve

Az MVC architektúra egyik legfontosabb célja, hogy elkülönítse az alkalmazás különböző felelősségeit. Ezáltal a program egyes részei egymástól függetlenül módosíthatók és tesztelhetők.

Az MVC három fő összetevőből áll:

- **Modell:** az adatok és a működési szabályok kezelése
- **Nézet:** az információk megjelenítése a felhasználó számára
- **Vezérlő:** a felhasználói interakciók feldolgozása

## Az MVC megjelenése a projektben

### Modell réteg

A projekt Modell részét elsősorban az `org.example.domain` csomag osztályai alkotják. Ide tartozik többek között a `Map`, a `Game`, a `Player`, a `ComputerPlayer`, a `ScoreBoard` és a `Configuration` osztály.

Ezek az osztályok felelnek:

- a játéktábla állapotának nyilvántartásáért,
- a lépések érvényességének ellenőrzéséért,
- a győzelmi feltételek vizsgálatáért,
- valamint az eredmények kezeléséért.

Fontos jellemzőjük, hogy nem kezelnek közvetlenül felhasználói bemenetet, ami megkönnyíti az egységtesztek írását és a kód ellenőrizhetőségét.

### Nézet réteg

Mivel az alkalmazás konzolos környezetben fut, a megjelenítés szöveges formában történik. A tábla kirajzolása és a játékállapot visszajelzései a konzolra történő kiíráson keresztül valósulnak meg.

A `Map.print()` metódus például kizárólag a tábla megjelenítéséért felel, így jól elkülönül a játékszabályokat tartalmazó logikától.

### Vezérlő szerep

A vezérlés feladatát a játék indításáért felelős fő osztály, valamint a `Game` osztály látja el. Ezek kezelik a felhasználói parancsokat, meghívják a megfelelő Modell metódusokat, és irányítják a játék menetét.

---

## 2. Gyártó (Factory) jellegű megoldások

### A minta lényege

A Factory jellegű megoldások célja, hogy az objektumok létrehozását elkülönítsük a felhasználásuktól. Ez nemcsak a kód olvashatóságát javítja, hanem csökkenti az osztályok közötti függőségeket is.

### Alkalmazás a projektben

Az `org.example.init` csomagban található osztályok ezt a szerepet töltik be:

- `MapInit`
- `PlayerInit`

Ezek az osztályok statikus metódusokon keresztül hozzák létre a játékhoz szükséges objektumokat. Például a `MapInit.createInitialMap()` metódus felel a játéktábla létrehozásáért és a kezdő játékos elhelyezéséért.

Ez a megközelítés lehetővé teszi, hogy a létrehozási logika egy helyen maradjon, és később könnyen módosítható legyen.

---

### 3. Stratégia jellegű megoldás a számítógépes játékosnál

#### Alapötlet

A Stratégia minta lényege, hogy egy viselkedést (például egy döntési algoritmust) elkülönítünk, és szükség esetén lecserélhetővé teszünk.

#### Megvalósítás

A projektben a `ComputerPlayer` osztály felel a számítógép lépéseinek kiválasztásáért. A `chooseMove()` metódus a lehetséges lépések közül választ egyet, jelenleg véletlenszerű módon.

Ez az egyszerű megoldás jól bővíthető: később könnyen bevezethető lenne egy összetettebb döntési algoritmus anélkül, hogy a játék többi részét át kellene írni.

---

### 4. Egyetlen felelősség elve (SRP)

A projekt több osztálya is jól példázza az egyetlen felelősség elvének alkalmazását:

- a `Map` kizárólag a tábla állapotával és szabályaival foglalkozik,
- a `ScoreBoard` csak az eredmények tárolásáért és megjelenítéséért felel,
- a `Configuration` a beállítások kezelését végzi.

Ez az elv hozzájárul ahhoz, hogy az osztályok egyszerűek, érthetők és könnyen tesztelhetők maradjanak.

---

## Összegzés

Az Amőba (Gomoku) projekt jó példát szolgáltat arra, hogy egy viszonylag egyszerű alkalmazás esetén is mennyire hasznosak a szoftvertervezési minták és architekturális elvek. Az MVC szerkezet átláthatóvá teszi a program felépítését, a Factory jellegű megoldások rendezik az objektumok létrehozását, míg a stratégia jellegű gondolkodásmód előkészíti a jövőbeli bővítések lehetőségét.

Ezek az elvek összességében hozzájárulnak ahhoz, hogy a kód karbantartható, bővíthető és megfelelően tesztelhető legyen.